

Noriko Takahashi

***This project is part of my broader research on near-real-time feedback for synthesis writing. Synthesis writing is a specific source-based writing task in which a writer identifies, connects, and integrates relevant main ideas from multiple source texts to respond to a writing prompt. It is not simply a summary of each source, and it is also different from rhetorical analysis, which focuses on how an author constructs an argument. In synthesis writing, the writer needs to explain relationships among ideas from different sources and organize them into one meaningful response.

Because these relationships often develop across several sentences, the timing of feedback is important. For example, if a writer produces a statement that conflicts with a fact in the source, the system may need to provide feedback immediately. However, the system cannot decide that a main idea or source connection is missing while the writer is still developing the current idea. Based on this feature of synthesis writing, I want to build a system that detects potential feedback triggers and decides whether to provide feedback immediately or wait until the writer finishes one idea chunk.***

General description of the project

- I want to make a writing feedback program for students who write “synthesis writing” from source texts. My main interest is not only what feedback should be given, but when it should appear. If feedback appears every time the system notices something, it could interrupt the writer too much.
- The program will use source texts that are already annotated. The annotations may include main ideas, supporting details, anecdotes/examples, inferences, and factual information. When the student writes something that changes or contradicts a fact from the source, the program should give feedback immediately. For example, it could catch a changed number, person, or cause-and-effect relationship.
- Not mentioning the main idea is different from writing a fact that conflicts with the source. The writer may start from an example and explain the main idea in the next sentence. Because of this, the system should wait until the writer finishes one idea chunk before giving feedback about the main idea or topic relevance. For Version 1, the user can type DONE to show that the current idea is finished.
- I may use JSON to store the annotations, sentence-transformers to find related parts of the source, and maybe a pretrained natural language inference model from transformers. I still need to test if the model is reliable enough. At a minimum, the program can run in the command line with one hardcoded source and sentence by sentence input.
- Ideally, this would later be a web based writing interface. It would track the text while the student writes, show immediate feedback only for a clear factual deviation, and wait to give other feedback until an idea is complete. I do not plan to make the full GUI for Version 1.

Task Vignettes (User activity flow)

- 1) Load the source and begin writing

the student has an assignment to write a response based on a short source text. She opens the program. For Version 1, the source may already be selected, so she does not need to upload anything. The program loads the source and its annotations, and then asks her to begin writing. the student enters one sentence at a time. The program stores each sentence as part of the current idea chunk. She can keep adding sentences until she decides that the idea is finished.

Details/ideas for later:

- Source and annotations could be stored in JSON or CSV.
- Each annotation needs the source span, its label, and an ID.
- Maybe let the user choose from several sources later.
- For Version 1, it is probably easier to hardcode one source filename.

2) Immediate feedback for a factual deviation

the student writes that a study had 1,200 participants, but the source says it had 120 participants. After she enters the sentence, the program finds the related sentence in the source and notices the different number. It gives feedback immediately, such as: This sentence may conflict with the source. The source reports 120 participants. The system should not call every new statement false. If something is not mentioned in the source, that only means it is unsupported by this source. It does not always mean it is incorrect. If the system is not confident, it should probably stay quiet or ask the student to check the source instead of making a strong correction.

Details/ideas for later:

- Input: the newest sentence as a string.
- Find the most related annotated source unit using sentence similarity.
- Possible output labels: consistent, conflicting, unsupported, and uncertain.
- Only a high-confidence conflict should interrupt the writer.
- Need to decide whether numbers/names should use separate rule-based checks.

3) Wait until one idea is finished

the student starts another part of her writing by describing an anecdote from the source. The first sentence does not mention the source's main idea. The program does not give feedback yet because the student may explain the connection in the next sentence. She writes another sentence and then types DONE. After DONE, the program checks the whole chunk. If the student connected the anecdote to the main idea, it does not need to say anything. If she only retold the anecdote, the program could ask: How does this example connect to the source's main point? The system can also check whether the whole chunk moved away from the source topic, although I am not sure yet how strict this check should be.

Details/ideas for later:

- Current idea chunk: a list of sentence strings.
- DONE is a temporary way to mark the idea boundary.
- Compare the complete chunk with annotated main ideas.
- Do not require every individual sentence to contain the main idea.
- Automatic idea-boundary detection would be a Version 2 feature.

4) End and review the writing session

When the student finishes writing, she enters END. The program shows her complete text and the feedback that appeared during the session. It also saves a record so I can check what caused immediate feedback and what caused delayed feedback.

Details/ideas for later:

- Save the sentence, chunk number, matched source unit, result, and feedback.
- JSON may be easier than CSV if one chunk contains several sentences.
- No login or multi-user database is needed for Version 1.
- Maybe later let the student accept, reject, or dismiss the feedback.

Technical flow

- The program loads one source text and its annotation file.
- The source annotations will probably be a list of dictionaries or a pandas DataFrame.
- The user enters one sentence through input().
- The sentence is compared with the annotated source units to find the most relevant unit.
- A factual consistency check returns a label and maybe a confidence score.
- If there is a clear conflict, the program prints immediate feedback.
- If there is no clear conflict, the sentence is added to the current idea chunk.
- When the user types DONE, the full chunk is compared with the annotated main ideas and other labels.
- The program prints delayed feedback only if the main-idea connection or relevance is missing.
- When the user types END, the text and feedback history are saved as JSON or CSV.

Possible blocks/functions:

- load_source() -> source text and annotations
- find_related_source_unit(sentence) -> source unit ID and similarity score
- check_factual_consistency(sentence, source_unit) -> label/confidence
- add_to_chunk(sentence) -> updated list of sentences
- check_finished_chunk(chunk) -> main-idea/relevance feedback
- save_session(records) -> local output file

Simple flow:

annotated source file -> load source -> user sentence -> find related source information -> factual check -> immediate feedback OR store sentence -> user types DONE -> chunk-level check -> delayed feedback -> save session

At a minimum for Version 1:

- Use one source and simulated/hardcoded annotations.
- Let the user write through the command line.
- Use DONE instead of trying to automatically detect an idea boundary.
- If the full NLI model is too difficult, start by detecting simpler factual changes such as numbers, names, and negation.

Unknowns/questions:

- Which pretrained NLI model works reasonably well with student writing?
- How high should the confidence be before interrupting the writer?
- How should the system separate unsupported information from an actual contradiction?
- Can main-idea connection be checked with sentence similarity, or will I need a different method?
- For the future GUI, how will the system know that one idea is finished without the DONE command?

Final self-assessment

- The biggest change from my first idea is that the system is now focused on feedback timing. I first thought mostly about detecting writing problems, but some feedback can be technically correct and still appear too early.
- I am confident that I can make the basic CLI, load annotations, collect sentences into chunks, and save the results. I am less confident about the accuracy of the factual consistency model.
- The biggest potential problem is false positive immediate feedback. If the system interrupts the writer and incorrectly says that a sentence conflicts with the source, the feedback will not be useful.
- I may need help choosing and testing an NLI model and deciding what is realistic for Version 1. I may also need to reduce the factual check to more specific cases if general contradiction detection does not work well.